

DAX Exercise 2

Intermediate DAX with the Northwind Dataset

Data Analytics | Power BI Desktop | Intermediate Level

Duration	Level	Tool	Dataset
60-75 min	Intermediate	Power BI Desktop	Northwind

1. Overview

In this exercise you will write intermediate DAX formulas against the Northwind sales data — the same dataset you transformed in Power Query Exercise 2. You will move beyond basic aggregations and learn patterns used daily by professional Power BI analysts: filtering with `CALCULATE`, calculating shares with `ALL` and `ALLSELECTED`, ranking with `RANKX`, cleaning up complex logic with `VAR...RETURN` and `SWITCH`, and performing year-over-year comparisons.

What You Will Learn

- bullet Review foundation measures on a multi-table star schema
- bullet Use `CALCULATE` with multiple filter conditions
- bullet Distinguish `ALL` from `ALLSELECTED` for % share calculations
- bullet Rank rows dynamically with `RANKX`
- bullet Write cleaner formulas using `VAR...RETURN`
- bullet Replace nested IF chains with `SWITCH`
- bullet Compare year-over-year revenue with `CALCULATE` and `DATEADD`
- bullet Use `SELECTEDVALUE` to build dynamic measure titles

Prerequisites

- bullet Completed Power Query Exercise 2 (Northwind dataset loaded and transformed)
- bullet Completed DAX Exercise 1 (familiar with Measures, Calculated Columns, `SUM`, `CALCULATE`, `IF`)
- bullet The `Northwind_Dataset.zip` CSV files available on your computer

Tables Used in This Exercise

Table	Key Column(s)	Role in This Exercise
OrderDetails	OrderID, ProductID	Fact table — one row per line item. Source of Quantity, UnitPrice, Discount.
Orders	OrderID, CustomerID, OrderDate, Freight	Links order-level info (date, customer, freight) to line items.
Products	ProductID, CategoryID	Brings in product name and category.
Categories	CategoryID, CategoryName	Dimension table for product category labels.
Customers	CustomerID, Country	Enables geographic analysis.
Employees	EmployeeID, Full Name	Enables sales-rep performance analysis.

2. Setup — Load Northwind and Build the Model

If you already completed Power Query Exercise 2 and saved your Power BI file, open it and proceed directly to Section 3. If starting fresh, follow the steps below.

Step 1 — Load the Six Northwind CSV Files

1. Open Power BI Desktop. Click Get data > Text/CSV.
2. Load all six files from Northwind_Dataset.zip — Orders, Order_Details, Products, Customers, Employees, Categories — using Transform Data for each.
3. In Power Query Editor, rename queries: Order_Details -> OrderDetails, remove spaces from all names.
4. In the OrderDetails query, add a custom column named Line Total:

Power Query -- Line Total column

```
= [UnitPrice] * [Quantity] * (1 - [Discount])
```

1. Set data types: Orders[OrderDate] -> Date, Orders[Freight] -> Decimal Number, OrderDetails[Quantity] -> Whole Number, OrderDetails[UnitPrice] and OrderDetails[Discount] -> Decimal Number.
2. In the Employees query, add a custom column: Full Name = [FirstName] & " " & [LastName].
3. Click Close & Apply.

Step 2 — Verify the Data Model

Switch to Model view and confirm these five relationships exist (create any that are missing by dragging the key column from one table to the other):

From (many side)	Key	To (one side)	Type
OrderDetails[OrderID]	OrderID	Orders[OrderID]	*:1
OrderDetails[ProductID]	ProductID	Products[ProductID]	*:1
Products[CategoryID]	CategoryID	Categories[CategoryID]	*:1
Orders[CustomerID]	CustomerID	Customers[CustomerID]	*:1
Orders[EmployeeID]	EmployeeID	Employees[EmployeeID]	*:1

Why This Model Shape Matters for DAX

This is a star schema: OrderDetails is the central fact table surrounded by dimension tables. DAX filters travel from dimension tables (Products, Customers, etc.) down into the fact table (OrderDetails) automatically.

You do NOT need RELATED to filter from a dimension into the fact table -- that happens via the relationship.

You DO need RELATED (or RELATEDTABLE) when a calculated column or SUMX needs to fetch a value from a related table row by row.

3. Task 1 — Foundation Measures

Build these baseline measures first. All later tasks reference them -- this is the measure-from-measure pattern that keeps DAX maintainable.

Where to Create Measures

Create all measures in the OrderDetails table unless otherwise stated.

Right-click OrderDetails in the Fields pane > New measure, or select the table and click New Measure in the Home ribbon.

Task 1A -- Total Revenue

DAX Measure

```
Total Revenue = SUM(OrderDetails[Line Total])
```

Task 1B -- Total Orders

Count distinct orders (not line items -- each order can have multiple products).

DAX Measure

```
Total Orders = DISTINCTCOUNT(OrderDetails[OrderID])
```

Task 1C -- Total Units Sold

DAX Measure

```
Total Units Sold = SUM(OrderDetails[Quantity])
```

Task 1D -- Average Order Value

Average revenue per order. Use DIVIDE to guard against zero orders.

DAX Measure

```
Avg Order Value = DIVIDE([Total Revenue], [Total Orders], 0)
```

Task 1E -- Total Freight

Freight lives in the Orders table. Because OrderDetails is the active table, DAX follows the relationship automatically.

DAX Measure

```
Total Freight = SUM(Orders[Freight])
```

Test All Five Measures

Add five Card visuals to the report canvas, one per measure.

Expected approximate values: Total Revenue ~\$1.3M, Total Orders ~830, Total Units Sold ~51,000, Avg Order Value ~\$1,600, Total Freight ~\$65,000.

Expected Result

Total Revenue: approx. \$1,265,000 - \$1,355,000

Total Orders: 830

Total Units Sold: approx. 51,000 - 53,000

Avg Order Value: approx. \$1,500 - \$1,650

Total Freight: approx. \$64,000 - \$66,000

4. Task 2 -- CALCULATE with Multiple Filters

In DAX Exercise 1 you used CALCULATE with a single filter. Real business questions often require two or more conditions applied at the same time.

CALCULATE -- Multi-Filter Syntax

```
// Multiple filter arguments act as AND conditions:
CALCULATE( <expression>, <filter1>, <filter2>, ... )

// To express OR conditions, use the IN operator or FILTER():
CALCULATE( <expression>, Products[Category] IN {"Beverages", "Seafood"} )
```

Task 2A -- Beverages Revenue

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
Beverages Revenue =
    CALCULATE(
        [Total Revenue],
        Categories[CategoryName] = "Beverages"
    )
```

Task 2B -- USA High-Value Orders

Revenue from orders shipped to the USA where Freight exceeded \$50. This requires two filter conditions.

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
USA High-Value Revenue =
    CALCULATE(
        [Total Revenue],
        Customers[Country] = "USA",
        Orders[Freight] > 50
    )
```

Task 2C -- Revenue Excluding Discounted Lines

Revenue from line items where no discount was applied. Use FILTER to express a row-level condition on the fact table.

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
Full-Price Revenue =  
    CALCULATE(  
        [Total Revenue],  
        FILTER(OrderDetails, OrderDetails[Discount] = 0)  
    )
```

CALCULATE filter vs FILTER()

Categories[CategoryName] = "Beverages" -- a simple column filter. Fast and preferred when filtering a single column value.

FILTER(table, <row condition>) -- row-by-row iteration. Use when the condition involves arithmetic or multiple columns in the same table (e.g. Discount = 0 on the fact table).

Mixing both styles in one CALCULATE is allowed and common.

Expected Result

Beverages Revenue: approx. \$260,000 - \$290,000

USA High-Value Revenue: varies by data -- roughly \$80,000 - \$120,000

Full-Price Revenue: approx. 60-70% of Total Revenue (discounted lines excluded)

5. Task 3 -- % Share: ALL vs ALLSELECTED

Two related but importantly different patterns for percentage-of-total calculations.

Function	Behaviour	Use When...
ALL(table)	Removes ALL filters -- denominator is always the grand total regardless of slicers	You want % of the entire dataset, slicer-independent
ALLSELECTED(table)	Removes filters from the visual context only, but KEEPS slicer selections	You want % within the current slicer selection (e.g. % within a chosen country)

Task 3A -- Category % of Grand Total (ALL)

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
Category % of Grand Total =  
    DIVIDE(  
        [Total Revenue],  
        CALCULATE([Total Revenue], ALL(Categories)),  
        0  
    ) * 100
```

Task 3B -- Category % of Selected Total (ALLSELECTED)

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
Category % of Selection =  
    DIVIDE(  
        [Total Revenue],  
        CALCULATE([Total Revenue], ALLSELECTED(Categories)),  
        0  
    ) * 100
```

See the Difference in Action

Step 1: Add a table visual with CategoryName, Total Revenue, Category % of Grand Total, and Category % of Selection.

Step 2: Add a slicer on Customers[Country] and select USA only.

Category % of Grand Total -- denominators still use ALL data. Percentages are small (share of global total).

Category % of Selection -- denominator is USA revenue only. Percentages sum to 100% within USA.

Both are valid -- the right choice depends on the business question.

Expected Result

Without slicer: both measures show identical percentages (e.g. Beverages ~21%, Dairy ~19%).

With USA slicer: Grand Total % stays small (e.g. Beverages USA ~6% of global); Selection % adds up to 100% within USA.

6. Task 4 -- RANKX

RANKX assigns a rank to each row based on the value of a measure, recalculating dynamically as filters change.

RANKX Syntax

```
RANKX( <table>, <expression>, [value], [order], [ties] )
```

```
table          -- the set of rows to rank across (usually ALL(table) or
VALUES(column))
expression     -- the measure to rank by
value          -- optional: the value for the current row (defaults to expression)
order          -- DESC (highest = 1, default) or ASC (lowest = 1)
ties           -- SKIP (1,2,2,4) or DENSE (1,2,2,3). Default is SKIP.
```

Task 4A -- Product Revenue Rank

Rank each product by its total revenue. Rank 1 = highest revenue.

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
Product Revenue Rank =
    RANKX(
        ALL(Products[ProductName]),
        [Total Revenue],
        ,
        DESC,
        DENSE
    )
```

Task 4B -- Category Revenue Rank

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
Category Revenue Rank =
    RANKX(
        ALL(Categories[CategoryName]),
        [Total Revenue],
        ,
        DESC,
        DENSE
    )
```

)

Test RANKX

Add a table visual: ProductName | Total Revenue | Product Revenue Rank.

Sort by Product Revenue Rank ascending. Rank 1 should be the product with the highest revenue.

Now apply a Category slicer and select Beverages only. Notice the ranks recalculate -- RANKX re-evaluates across ALL products, so the Beverages products are ranked 1, 2, 3 relative to all 77 products globally.

Change ALL(Products[ProductName]) to ALLSELECTED(Products[ProductName]) and observe how ranks reset to 1, 2, 3 within the current slicer selection.

Expected Result

Rank 1 product: Cote de Blaye (the most expensive product in the dataset).

Rank 1 category: Beverages (largest revenue category in Northwind).

With DENSE ties: no gaps in ranking even when two products tie on revenue.

7. Task 5 -- VAR...RETURN for Cleaner Formulas

VAR...RETURN lets you store intermediate results as named variables inside a DAX formula. This makes complex measures far easier to read, debug, and maintain -- and can also improve performance by avoiding repeated evaluation of the same sub-expression.

VAR...RETURN Syntax

```
MeasureName =  
    VAR variable1 = <expression1>  
    VAR variable2 = <expression2>  
    RETURN  
        <final expression using variable1 and variable2>
```

Task 5A -- Rewrite Avg Order Value Using VAR

The original Avg Order Value measure works, but rewriting it with VAR makes the logic explicit and self-documenting.

1. Open the existing Avg Order Value measure (click it in the Fields pane).
2. Replace the formula with the version below.

DAX Measure -- Refactored with VAR

```
Avg Order Value =  
    VAR TotalRev    = [Total Revenue]  
    VAR TotalOrds   = [Total Orders]  
    RETURN  
        DIVIDE(TotalRev, TotalOrds, 0)
```

Task 5B -- Gross Profit with VAR

Gross Profit requires subtracting total cost from total revenue. Using SUMX and RELATED to calculate cost, and VAR to store both sides, keeps the formula clean.

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure -- Gross Profit with VAR

```
Gross Profit =  
    VAR TotalRevenue =  
        SUM(OrderDetails[Line Total])  
    VAR TotalCost =  
        SUMX(  
            OrderDetails,  
            OrderDetails[Quantity] * RELATED(Products[UnitPrice]) * 0.6  
        )  
    RETURN  
        TotalRevenue - TotalCost  
  
// Note: 0.6 is an assumed 60% cost ratio since Northwind has no CostPrice column.  
// In a real dataset you would use the actual cost column.
```

Task 5C -- Profit Margin % with VAR

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure

```
Profit Margin % =  
    VAR Profit    = [Gross Profit]  
    VAR Revenue   = [Total Revenue]  
    RETURN  
        DIVIDE(Profit, Revenue, 0) * 100
```

Expected Result

Gross Profit: approx. \$500,000 - \$560,000 (40% of Total Revenue, based on 60% cost assumption).

Profit Margin %: approx. 40% at overall level.

Both measures respond correctly to slicers -- VAR captures the filtered context at evaluation time.

8. Task 6 -- SWITCH for Readable Conditional Logic

SWITCH replaces deeply nested IF chains with a flat, readable list of cases. It is the DAX equivalent of a CASE statement in SQL.

SWITCH Syntax

```
// SWITCH with explicit values:
SWITCH( <expression>, <value1>, <result1>, <value2>, <result2>, ..., [else] )

// SWITCH(TRUE()) -- evaluates each condition in order, returns first match:
SWITCH(
    TRUE(),
    <condition1>, <result1>,
    <condition2>, <result2>,
    <else>
)
```

Task 6A -- Order Size Category

Assign each order line item a size label based on the Line Total value.

1. In Data view, select the OrderDetails table.
2. Click New Column in the Home ribbon.
3. Enter the formula below.

DAX Calculated Column -- Order Size

```
Order Size =
    SWITCH(
        TRUE(),
        OrderDetails[Line Total] >= 2000, "Large",
        OrderDetails[Line Total] >= 500, "Medium",
        OrderDetails[Line Total] >= 100, "Small",
        "Micro"
    )
```

Task 6B -- Freight Band

Classify each order's freight cost into a band. Freight lives in the Orders table, so use RELATED to access it from OrderDetails.

1. Create a new calculated column in OrderDetails.
2. Enter the formula below.

DAX Calculated Column -- Freight Band (SWITCH + VAR + RELATED)

```
Freight Band =  
    VAR F = RELATED(Orders[Freight])  
    RETURN  
        SWITCH(  
            TRUE(),  
            F >= 100, "High",  
            F >= 30, "Medium",  
            "Low"  
        )
```

Why SWITCH(TRUE()) Instead of IF?

Nested IF: IF(a, x, IF(b, y, IF(c, z, w))) -- hard to read beyond two levels.

SWITCH(TRUE()): each line is one condition + result -- easy to add, remove, or reorder cases.

Both produce identical results. In production DAX code SWITCH(TRUE()) is the professional standard.

Expected Result

Order Size column: four distinct values -- Large, Medium, Small, Micro -- across 2,470 rows.

Freight Band column: three values -- High, Medium, Low.

Add a matrix visual: Order Size on rows, Freight Band on columns, Total Revenue as values.

9. Task 7 -- Year-Over-Year Comparison

The Northwind orders span three calendar years: 1996, 1997, and 1998. In this task you will build measures to compare revenue across years and calculate year-over-year growth.

About the Northwind Dates

Orders date range: July 1996 through May 1998.

1996 is a partial year (6 months). 1998 is also partial (5 months).

1997 is the only complete year in the dataset -- keep this in mind when interpreting growth rates.

For a full time-intelligence implementation in production you would build a dedicated Date table. In this exercise we use CALCULATE with YEAR() for simplicity.

Task 7A -- Revenue by Year

Create one measure per year using CALCULATE filtered by YEAR(Orders[OrderDate]).

1. Create three new measures in OrderDetails.
2. Enter the three formulas below.

DAX Measures -- Revenue by Year

```
Revenue 1996 =  
    CALCULATE(  
        [Total Revenue],  
        YEAR(Orders[OrderDate]) = 1996  
    )  
  
Revenue 1997 =  
    CALCULATE(  
        [Total Revenue],  
        YEAR(Orders[OrderDate]) = 1997  
    )  
  
Revenue 1998 =  
    CALCULATE(  
        [Total Revenue],  
        YEAR(Orders[OrderDate]) = 1998  
    )
```

Task 7B -- Year-Over-Year Growth % (1996 to 1997)

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure -- YoY Growth %

```
YoY Growth % 96-97 =  
    VAR Rev96 = [Revenue 1996]  
    VAR Rev97 = [Revenue 1997]  
    RETURN  
        DIVIDE(Rev97 - Rev96, Rev96, 0) * 100
```

Task 7C -- Year-to-Date Revenue (Dynamic)

TOTALYTD calculates the running total from January 1 to the current date in the filter context. It requires a Date column.

1. Create a new measure in OrderDetails.
2. Enter the formula below.

DAX Measure -- Revenue YTD

```
Revenue YTD =  
    TOTALYTD(  
        [Total Revenue],  
        Orders[OrderDate]  
    )  
  
// TOTALYTD is equivalent to:  
// CALCULATE([Total Revenue],  
//     DATESYTD(Orders[OrderDate]))
```

Test the YTD Measure

Add a table visual with YEAR(Orders[OrderDate]) and Revenue YTD.

The YTD resets to 0 at the start of each year and accumulates through the year.

In a matrix with Month on rows and Year on columns, Revenue YTD shows the cumulative total month by month.

Expected Result

Revenue 1996: approx. \$200,000 - \$230,000 (Jul-Dec only)

Revenue 1997: approx. \$650,000 - \$720,000 (full year)

Revenue 1998: approx. \$400,000 - \$460,000 (Jan-May only)

YoY Growth 96-97: approx. +180% to +230% (large because 1996 is only half a year)

10. Task 8 -- SELECTEDVALUE for Dynamic Labels

SELECTEDVALUE returns the single selected value of a column when exactly one item is in the filter context -- for example, when a slicer has one item chosen. When multiple values or no values are selected it returns a default result. This lets you build dynamic report titles and labels.

SELECTEDVALUE Syntax

```
SELECTEDVALUE( <column>, [alternateResult] )
```

Returns the column value if exactly 1 value is in context.

Returns alternateResult if 0 or more than 1 value is in context.

Task 8A -- Dynamic Report Title

1. Create a new measure in OrderDetails.

2. Enter the formula below.

DAX Measure -- Dynamic Report Title

```
Report Title =  
    VAR SelectedCountry =  
        SELECTEDVALUE (Customers[Country], "All Countries")  
    VAR SelectedCategory =  
        SELECTEDVALUE (Categories[CategoryName], "All Categories")  
    RETURN  
        "Revenue | " & SelectedCountry & " | " & SelectedCategory
```

1. Add a Card visual to the report canvas and place Report Title on it.
2. Add a Country slicer and a Category slicer.
3. Select Germany in the Country slicer. The card updates to: Revenue | Germany | All Categories.
4. Now also select Beverages in the Category slicer. The card reads: Revenue | Germany | Beverages.
5. Deselect both slicers -- the card returns to: Revenue | All Countries | All Categories.

Expected Result

Card shows 'Revenue | All Countries | All Categories' with no slicers active.
Card updates dynamically to reflect any single slicer selection.
When multiple countries are selected the card reverts to 'All Countries'.

11. Building a Summary Dashboard

Now that all measures are built, assemble them into a one-page summary dashboard.
Suggested layout:

Visual Type	Fields	Purpose
Card (x5)	Total Revenue, Total Orders, Total Units Sold, Avg Order Value, Profit Margin %	KPI summary row at the top
Card (title)	Report Title	Dynamic title driven by slicers
Bar chart	CategoryName Total Revenue + Category % of Grand Total	Revenue and share by category

Table	CategoryName Total Revenue Category Revenue Rank Gross Profit Profit Margin %	Category performance table
Clustered column	Year Revenue 1996, 1997, 1998	Year-over-year comparison
Matrix	Order Size (rows) Freight Band (cols) Total Revenue (values)	Order segmentation
Slicer	Customers[Country]	Geographic filter
Slicer	Categories[CategoryName]	Category filter

12. Solutions -- All DAX Formulas

Complete reference of every measure and calculated column from this exercise.

All Measures (OrderDetails table)

Complete Measures

```
// — TASK 1: FOUNDATION —————
Total Revenue      = SUM(OrderDetails[Line Total])
Total Orders       = DISTINCTCOUNT(OrderDetails[OrderID])
Total Units Sold   = SUM(OrderDetails[Quantity])
Total Freight      = SUM(Orders[Freight])
Avg Order Value    =
    VAR TotalRev = [Total Revenue]
    VAR TotalOrds = [Total Orders]
    RETURN DIVIDE(TotalRev, TotalOrds, 0)

// — TASK 2: CALCULATE WITH MULTIPLE FILTERS —————
Beverages Revenue =
    CALCULATE([Total Revenue], Categories[CategoryName] = "Beverages")

USA High-Value Revenue =
    CALCULATE([Total Revenue], Customers[Country] = "USA", Orders[Freight] > 50)

Full-Price Revenue =
    CALCULATE([Total Revenue], FILTER(OrderDetails, OrderDetails[Discount] = 0))

// — TASK 3: % SHARE —————
Category % of Grand Total =
    DIVIDE([Total Revenue], CALCULATE([Total Revenue], ALL(Categories)), 0) * 100
```

```

Category % of Selection =
    DIVIDE([Total Revenue], CALCULATE([Total Revenue], ALLSELECTED(Categories)),
0) * 100

// — TASK 4: RANKX —————
Product Revenue Rank =
    RANKX(ALL(Products[ProductName]), [Total Revenue], , DESC, DENSE)

Category Revenue Rank =
    RANKX(ALL(Categories[CategoryName]), [Total Revenue], , DESC, DENSE)

// — TASK 5: VAR...RETURN —————
Gross Profit =
    VAR TotalRevenue = SUM(OrderDetails[Line Total])
    VAR TotalCost    = SUMX(OrderDetails,
                            OrderDetails[Quantity] * RELATED(Products[UnitPrice]) *
0.6)
    RETURN TotalRevenue - TotalCost

Profit Margin % =
    VAR Profit  = [Gross Profit]
    VAR Revenue = [Total Revenue]
    RETURN DIVIDE(Profit, Revenue, 0) * 100

// — TASK 7: YEAR-OVER-YEAR —————
Revenue 1996 = CALCULATE([Total Revenue], YEAR(Orders[OrderDate]) = 1996)
Revenue 1997 = CALCULATE([Total Revenue], YEAR(Orders[OrderDate]) = 1997)
Revenue 1998 = CALCULATE([Total Revenue], YEAR(Orders[OrderDate]) = 1998)

YoY Growth % 96-97 =
    VAR Rev96 = [Revenue 1996]
    VAR Rev97 = [Revenue 1997]
    RETURN DIVIDE(Rev97 - Rev96, Rev96, 0) * 100

Revenue YTD = TOTALYTD([Total Revenue], Orders[OrderDate])

// — TASK 8: SELECTEDVALUE —————
Report Title =
    VAR SelectedCountry = SELECTEDVALUE(Customers[Country], "All Countries")
    VAR SelectedCategory = SELECTEDVALUE(Categories[CategoryName], "All
Categories")
    RETURN "Revenue | " & SelectedCountry & " | " & SelectedCategory

```

All Calculated Columns (OrderDetails table)

Complete Calculated Columns

```

// — TASK 6A: Order Size —————
Order Size =
    SWITCH(TRUE(),

```

```

        OrderDetails[Line Total] >= 2000, "Large",
        OrderDetails[Line Total] >= 500,  "Medium",
        OrderDetails[Line Total] >= 100,  "Small",
        "Micro"
    )

// — TASK 6B: Freight Band —————
Freight Band =
    VAR F = RELATED(Orders[Freight])
    RETURN
        SWITCH(TRUE(),
            F >= 100, "High",
            F >= 30,  "Medium",
            "Low"
        )

```

Key Function Quick Reference

Function / Pattern	Purpose
CALCULATE(expr, f1, f2)	Evaluate expr with multiple filter overrides (AND logic)
CALCULATE(expr, col IN {a,b})	Evaluate expr with OR filter across values
FILTER(table, condition)	Row-level filter inside CALCULATE
ALL(table)	Remove all filters -- grand-total denominator
ALLSELECTED(table)	Remove visual filters but keep slicer filters
RANKX(ALL(col), measure)	Dynamic rank across all values of a column
VAR x = expr RETURN	Store sub-results; improve readability and performance
SWITCH(TRUE(), cond, val)	Flat conditional logic -- replaces nested IF
RELATED(col)	Fetch value from related table (row context)
TOTALYTD(expr, dateCol)	Cumulative total from Jan 1 to current date
SELECTEDVALUE(col, alt)	Single selected value from slicer; alt if 0 or many

13. Reflection Questions

Answer the following questions based on your work in this exercise.

1. In Task 2B you used two filter arguments in CALCULATE: Customers[Country] = "USA" and Orders[Freight] > 50. Are these filters joined by AND or OR? How would you change the formula to return revenue where the country is USA OR Germany?

2. Explain in your own words the difference between ALL(Categories) and ALLSELECTED(Categories) in a % share measure. When would a business analyst choose one over the other?

3. In Task 4, RANKX uses ALL(Products[ProductName]) in its table argument. What would happen to the ranking if you used VALUES(Products[ProductName]) instead? When would VALUES be the right choice?

4. The VAR...RETURN pattern was introduced in Task 5. Beyond readability, VAR has a performance benefit: each variable is evaluated only once even if referenced multiple times. Rewrite the YoY Growth % measure without VAR. What disadvantage does the non-VAR version have?

5. The Northwind dataset has no CostPrice column, so you used a 60% cost ratio assumption in the Gross Profit measure. What are the risks of this approach? How would you improve it if a real CostPrice column were added to the Products table?

6. In Task 8, SELECTEDVALUE returns 'All Countries' when multiple countries are selected in a slicer. A colleague asks: 'Can I make the card show a comma-separated list of selected countries instead?' Describe the DAX approach you would use (function name and general logic -- no need to write the full formula).

Suggested Answers

1. Multiple CALCULATE filters

Multiple filter arguments in CALCULATE are AND conditions -- both must be true for a row to be included. To use OR logic for countries you would write: `CALCULATE([Total Revenue], Customers[Country] IN {"USA", "Germany"})` using the IN operator, or `CALCULATE([Total Revenue], OR(Customers[Country] = "USA", Customers[Country] = "Germany"))`.

2. ALL vs ALLSELECTED

ALL removes every filter on the table, so the denominator is always the entire dataset regardless of slicers -- useful when you want 'what % of global total is this category?' ALLSELECTED removes only the filter from the current visual (e.g. the table row filter) but keeps the slicer selection, so the denominator is 'total within the slicer' -- useful when you want 'what % of the selected subset is this category?' A business analyst would choose ALLSELECTED when users expect percentages to re-base when they filter, and ALL when percentages should always be relative to the full dataset.

3. ALL vs VALUES in RANKX

ALL(Products[ProductName]) ranks the current product against all 77 products regardless of any slicer. VALUES(Products[ProductName]) ranks only against the products currently visible in the filter context. Use ALL for absolute global rank ('this product is 3rd overall'); use VALUES when you want rank within the current selection ('this product is 1st among the filtered set').

4. VAR vs no-VAR in YoY

Without VAR: `DIVIDE([Revenue 1997] - [Revenue 1996], [Revenue 1996], 0) * 100` -- each measure reference ([Revenue 1996], [Revenue 1997]) is evaluated separately. If [Revenue 1996] is referenced twice (numerator and denominator) it is evaluated twice, which is inefficient in a large model. It is also harder to debug because you cannot inspect intermediate values.

5. 60% cost assumption risk

The 60% assumption applies a uniform margin to all products, which is unrealistic -- a bottle of wine and a box of paper clips have very different margins. This understates profit for high-margin products and overstates it for low-margin ones. If a real CostPrice were added, you would replace `'RELATED(Products[UnitPrice]) * 0.6'` with `'RELATED(Products[CostPrice])'` for accurate per-product cost.

6. Comma-separated list with CONCATENATEX

Use `CONCATENATEX(VALUES(Customers[Country]), Customers[Country], ", ")` which iterates over all currently selected country values and joins them with a comma separator. This can be wrapped in a VAR inside the Report Title measure.

14. Challenge Extension

Completed all tasks? These challenges require combining multiple concepts from this exercise.

- A** **Top 5 Products by Revenue**
Write a measure that returns 1 if a product is in the top 5 by revenue, and 0 otherwise. Use it as a visual-level filter to show only top-5 products in a bar chart.
`Top 5 Flag = IF([Product Revenue Rank] <= 5, 1, 0)`
- B** **Country Revenue Share with Running Total**
Create a table with Country, Total Revenue, Category % of Grand Total (renamed as Country % of Total), and a running-total column.
Hint: use RANKX on Customers[Country] to order rows, then CALCULATE with a row filter using that rank to accumulate revenue.
- C** **1997 vs 1998 Partial-Year Comparison**
Because 1998 only has data through May, a direct year comparison is unfair. Write a measure for 'Revenue Jan-May 1997' to create an apples-to-apples comparison with 1998:
`Revenue Jan-May 1997 = CALCULATE([Total Revenue],
YEAR(Orders[OrderDate]) = 1997,
MONTH(Orders[OrderDate]) <= 5)`
Then calculate the true YoY growth between these two comparable periods.
- D** **Employee Performance Dashboard**
Build a table showing each employee (Full Name), their Total Revenue, Product Revenue Rank (rewritten as Employee Revenue Rank), and a Performance Label using SWITCH:
`Top Performer if rank = 1
Strong if rank <= 3
Developing otherwise`
Add a Card for the top performer's name using SELECTEDVALUE after filtering by rank = 1.
- E** **Discount Impact Analysis**
Create two measures: Discounted Revenue (lines with Discount > 0) and Discount Savings (how much revenue was lost to discounts).
`Discount Savings = SUMX(OrderDetails,
OrderDetails[Quantity] * OrderDetails[UnitPrice] *
OrderDetails[Discount])`
Visualise what % of potential revenue was given away as discounts per category.